

## Ex. No: 04

### Image Translation Using OpenCV

**\*\*Name : \*\*ASWINI D Reg. No : 212225240015 Slot No : OD1199**

## Image Translation

Write a Python program using OpenCV to perform image translation on the **"Given image"**.

The program should:

1. Import `cv2` , `numpy` , and `matplotlib.pyplot` .
2. Read the image **"given"** in color mode.
3. Create a **translation matrix** that moves the image **50 pixels to the right** and **80 pixels down**.
4. Apply the translation
5. Display the following using a **1 × 2 layout** in Matplotlib:
  - **Original Image**
  - **Translated Image**

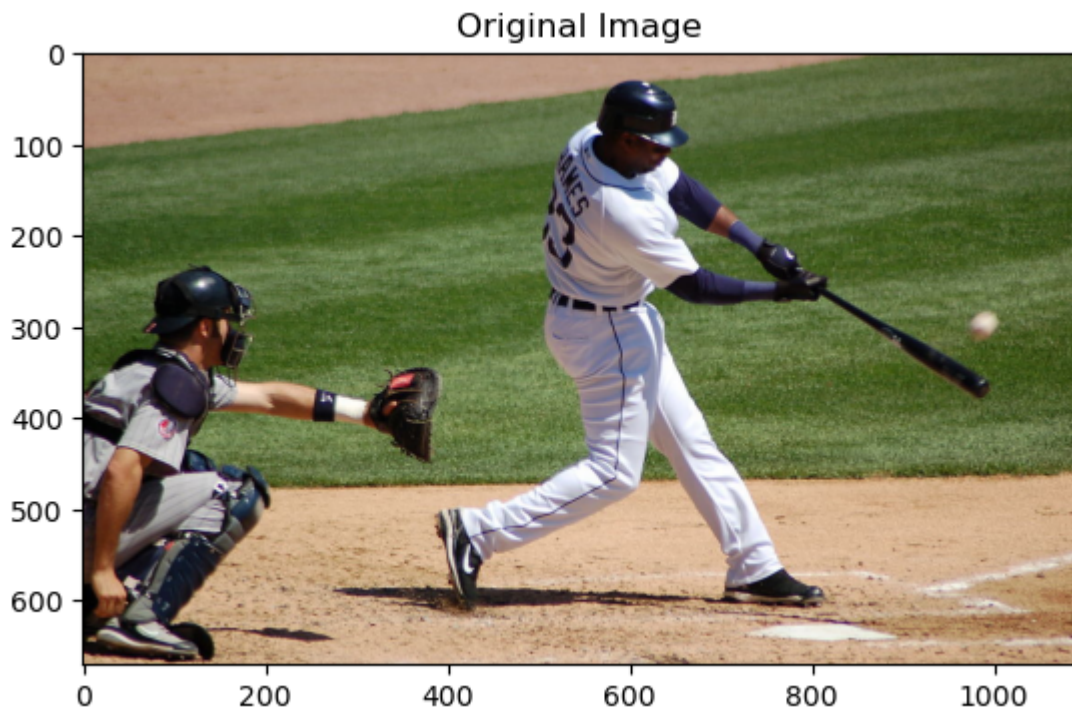
```
In [1]: # Import
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
In [11]: # Read the image using OpenCV
image = cv2.imread('baseball.jpg')
image.shape
```

```
Out[11]: (671, 1090, 3)
```

```
In [10]: # Display the image using Matplotlib

plt.imshow(image[:,:,:-1])
plt.title('Original Image')
plt.show()
```



## Image Scaling

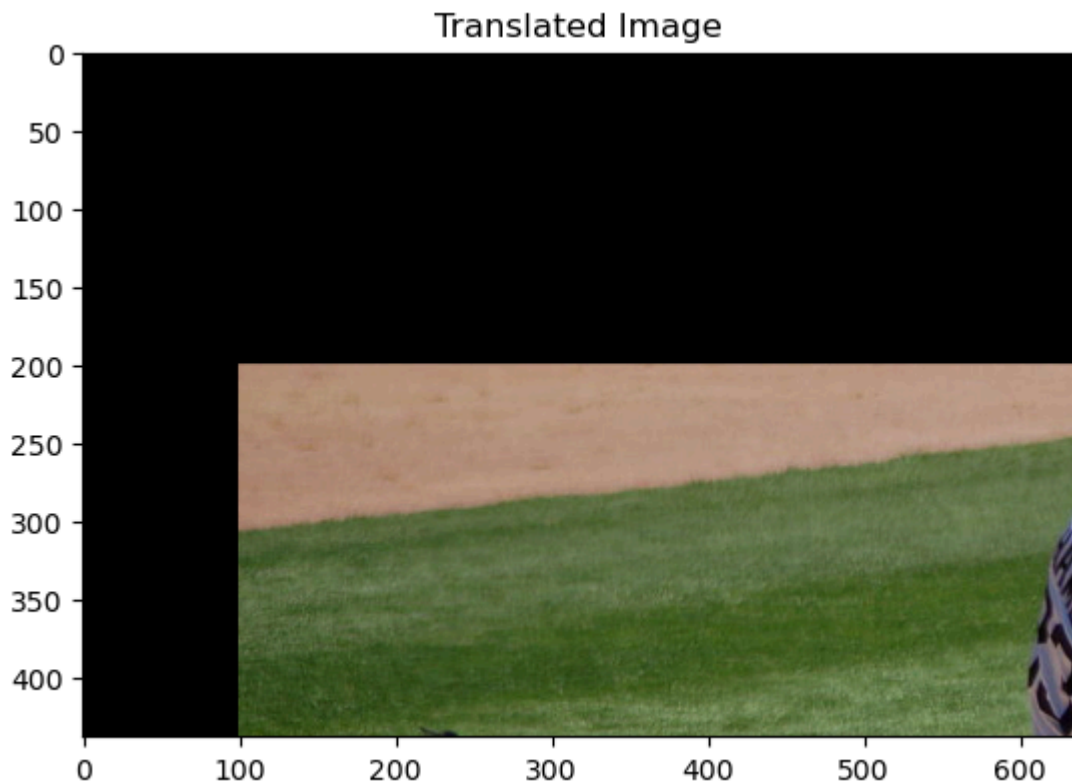
Write a Python program using OpenCV to perform image scaling on the **given image**.

The program should:

1. Scale the image to:
  - **0.5 times** its original size (downscaling)
  - **2 times** its original size (upscaling)
2. Use `cv2.resize` with appropriate scaling factors.
3. Display the following using a **1 × 3 layout** in Matplotlib:
  - **Original Image**
  - **Downscaled Image (0.5×)**
  - **Upscaled Image (2×)**

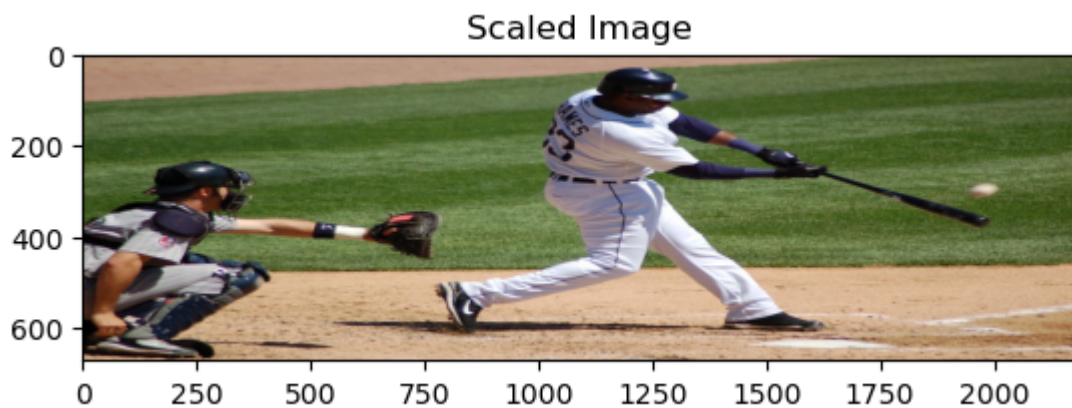
```
In [12]: tx, ty = 100, 200
M_translation = np.float32([[1, 0, tx], [0, 1, ty]])
translated_image = cv2.warpAffine(image, M_translation, (636, 438))
```

```
In [13]: plt.imshow(translated_image[:, :, ::-1])
plt.title("Translated Image")
plt.axis('on')
plt.show()
```



```
In [14]: fx, fy = 2.0, 1.0
scaled_image = cv2.resize(image, None, fx=fx, fy=fy, interpolation=cv2.INTER_LINEAR)
```

```
In [15]: plt.imshow(scaled_image[:, :, ::-1])
plt.title("Scaled Image")
plt.axis('on')
plt.show()
```



## Image Shearing

Write a Python program using OpenCV to perform image shearing on the **given image**.

The program should:

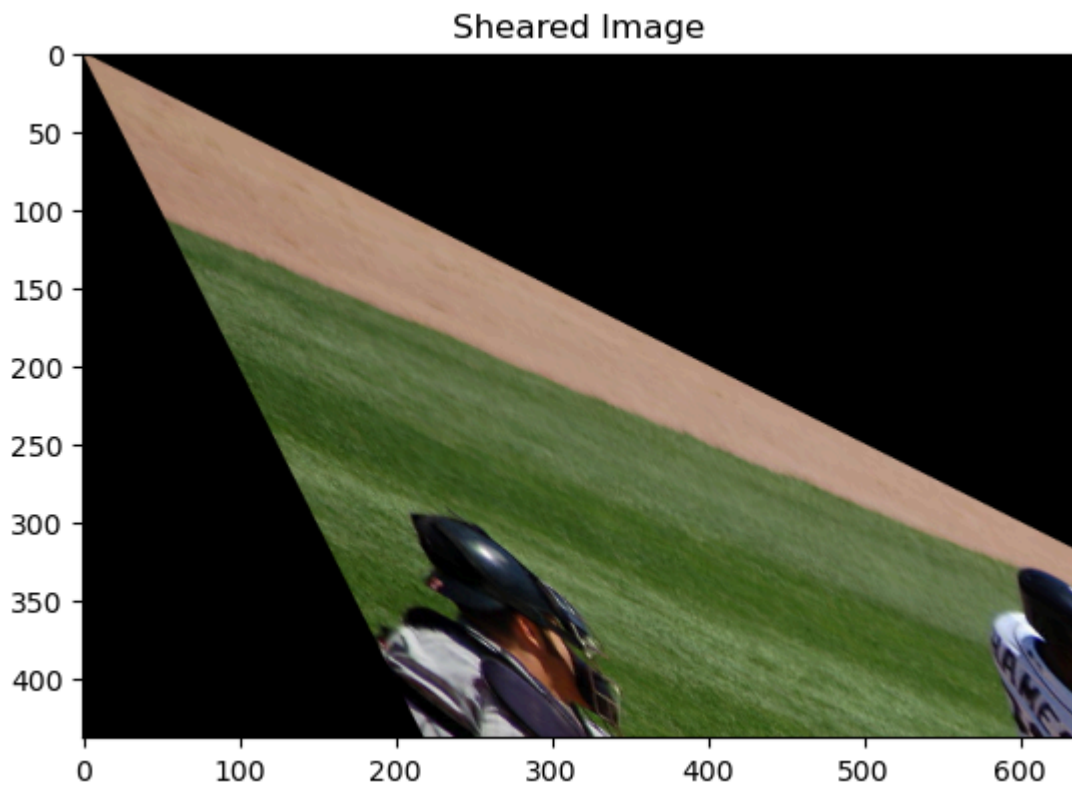
1. Create a **shearing transformation matrix** to apply:
  - **Horizontal shearing**
  - **Vertical shearing**
2. Apply the shearing transformations.

3. Display the following using a **1 × 3 layout** in Matplotlib:

- **Original Image**
- **Horizontally Sheared Image**
- **Vertically Sheared Image**

```
In [16]: shear_matrix = np.float32([[1, 0.5, 0], [0.5, 1, 0]])  
sheared_image = cv2.warpAffine(image, shear_matrix, (636, 438))
```

```
In [17]: plt.imshow(sheared_image[:, :, ::-1])  
plt.title("Sheared Image")  
plt.axis('on')  
plt.show()  
reflected_image = cv2.flip(image, 2)  
plt.figure(figsize=(10, 5))
```



```
Out[17]: <Figure size 1000x500 with 0 Axes>
```

```
<Figure size 1000x500 with 0 Axes>
```

```
In [18]: plt.subplot(1, 2, 1)  
plt.imshow(image[:, :, ::-1])  
plt.title("Original Image")  
plt.axis('off')
```

```
Out[18]: (np.float64(-0.5), np.float64(1089.5), np.float64(670.5), np.float64(-0.5))
```

Original Image



## Image Reflection

Write a Python program using OpenCV to perform image reflection on the **given image**.

The program should:

1. Generate the following reflected versions of the image:
  - **Horizontal Reflection** (flip left ↔ right)
  - **Vertical Reflection** (flip top ↔ bottom)
  - **Both-axis Reflection**
2. Use `cv2.flip` with appropriate flip codes.
3. Display the following using a **2 × 2 layout** in Matplotlib:
  - **Original Image**
  - **Horizontally Reflected Image**
  - **Vertically Reflected Image**
  - **Both-axis Reflected Image**

```
In [19]: plt.subplot(1, 2, 2)
plt.imshow(reflected_image[:, ::-1])
plt.title("Reflected Image")
plt.axis('off')
```

```
Out[19]: (np.float64(-0.5), np.float64(1089.5), np.float64(670.5), np.float64(-0.5))
```

Reflected Image



```
In [20]: plt.tight_layout()
plt.show()
(height, width) = image.shape[:2]
angle = 45
center = (width // 2, height // 2)
```

<Figure size 640x480 with 0 Axes>

```
In [21]: M_rotation = cv2.getRotationMatrix2D(center, angle, 1)
rotated_image = cv2.warpAffine(image, M_rotation, (width, height))
plt.imshow(cv2.cvtColor(rotated_image, cv2.COLOR_BGR2RGB))
plt.title("Rotated Image")
plt.axis('off')
```

```
Out[21]: (np.float64(-0.5), np.float64(1089.5), np.float64(670.5), np.float64(-0.5))
```

Rotated Image



```
In [22]:
```

```
In [ ]:
```

```
In [ ]:
```

## Image Rotation

Write a Python program using OpenCV to perform **image rotation** on the **given image**.

The program should:

1. Rotate the image by:
  - **45 degrees**
  - **90 degrees**
2. Use `cv2.getRotationMatrix2D` and `cv2.warpAffine`.
3. Display the following using a **1 × 3 layout** in Matplotlib:
  - **Original Image**
  - **45° Rotated Image**
  - **90° Rotated Image**

```
In [23]: image.shape
angle = 145
center = (636 // 2, 438 // 2)
```

```
M_rotation = cv2.getRotationMatrix2D(center, angle, 1)
rotated_image = cv2.warpAffine(image, M_rotation, (width, height))
```

```
In [24]: plt.imshow(rotated_image[:, :, ::-1]) # Display the rotated image
plt.title("Rotated Image") # Set title
plt.axis('off')
plt.show()
# Image Cropping
x, y, w, h = 0, 0, 200, 150
```

Rotated Image



```
In [25]: cropped_image = image[y:y+h, x:x+w]
plt.figure(figsize=(10, 5))
```

```
Out[25]: <Figure size 1000x500 with 0 Axes>
<Figure size 1000x500 with 0 Axes>
```

```
In [26]: plt.subplot(1, 2, 1)
plt.imshow(image[:, :, ::-1])
plt.title("Original Image")
plt.axis('on')
```

```
Out[26]: (np.float64(-0.5), np.float64(1089.5), np.float64(670.5), np.float64(-0.5))
```

Original Image



```
In [29]: plt.subplot(1, 2, 2)
plt.imshow(cropped_image[:, :, ::-1])
plt.title("Cropped Image")
```

```
plt.axis('on')  
plt.tight_layout()  
plt.show()
```

